

COGNIZANT

Mandatory Hands-on Exercises

-Khushi Ahi

TOPIC: Spring Data JPA with Hibernate

Exercise 1: Spring Data JPA - Quick Example

CODE

```
MySQL 8.0 Command Line Cli  X  +  v
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.34 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ^C
mysql> CREATE DATABASE ormllearn;
Query OK, 1 row affected (0.08 sec)

mysql> USE ormllearn;
Database changed
mysql> CREATE TABLE country (
    ->     co_code VARCHAR(2) PRIMARY KEY,
    ->     co_name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO country VALUES ('IN', 'India');
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO country VALUES ('US', 'United States of America');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM country;
+-----+-----+
| co_code | co_name                |
+-----+-----+
| IN      | India                   |
| US      | United States of America |
+-----+-----+
2 rows in set (0.01 sec)

mysql> |
```

Spring Initializr

start.spring.io

Project

Language

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web

Spring Data JPA

MySQL Driver

Project Metadata

Group

Artifact

Package name

Packaging

Configuration

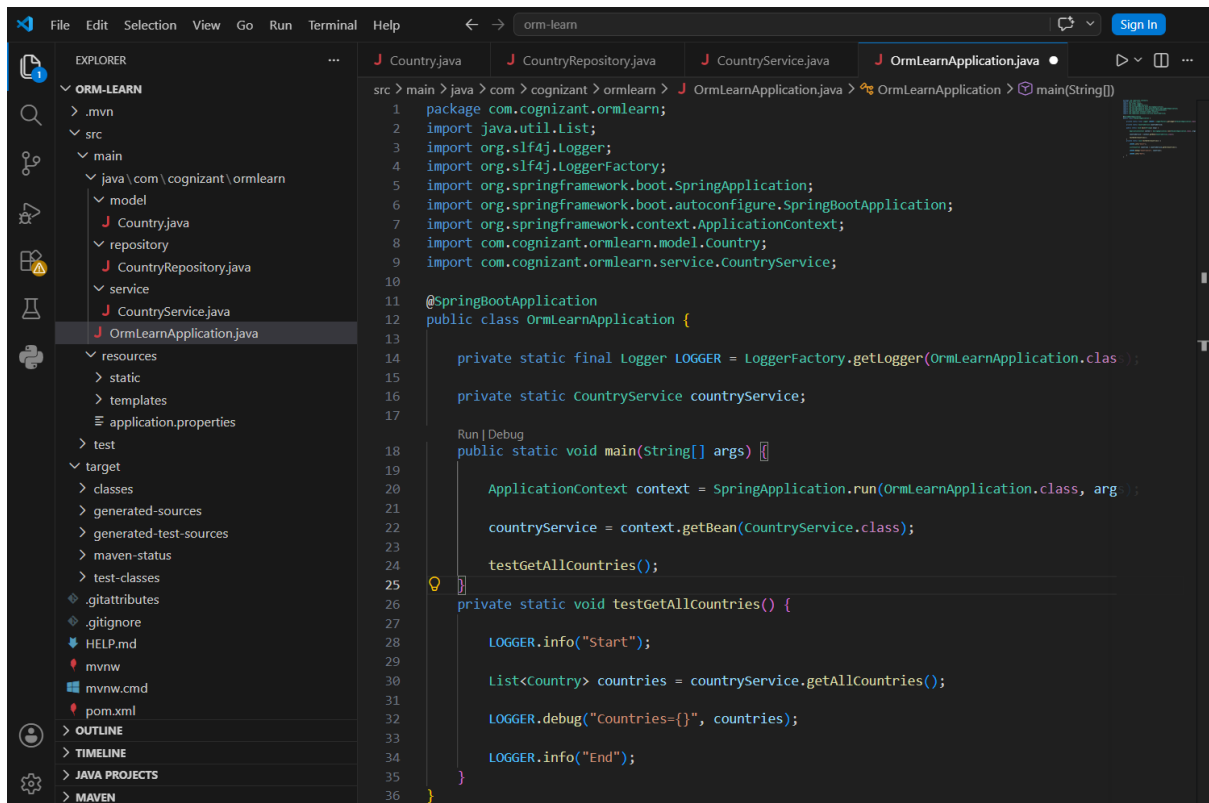
Java

GENERATE CTRL + G

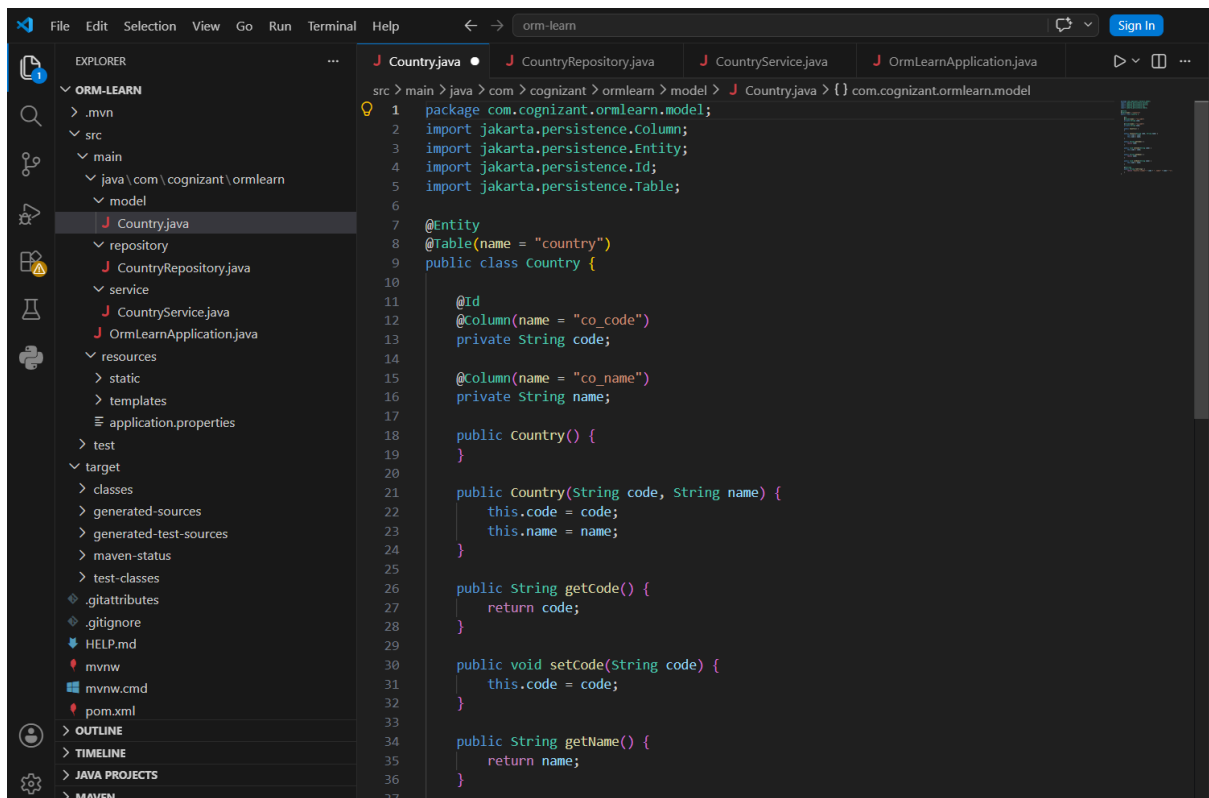
EXPLORE CTRL + SPACE

```
src > main > java > com > cognizant > ormlearn > repository > CountryRepository
1 package com.cognizant.ormlearn.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5
6 import com.cognizant.ormlearn.model.Country;
7
8 @Repository
9 public interface CountryRepository extends JpaRepository<Country, String> {
10
11 }
```

```
src > main > java > com > cognizant > ormlearn > service > CountryService
1 package com.cognizant.ormlearn.service;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7 import org.springframework.transaction.annotation.Transactional;
8
9 import com.cognizant.ormlearn.model.Country;
10 import com.cognizant.ormlearn.repository.CountryRepository;
11
12 @Service
13 public class CountryService {
14
15     @Autowired
16     private CountryRepository countryRepository;
17
18     @Transactional
19     public List<Country> getAllCountries() {
20         return countryRepository.findAll();
21     }
22 }
```



```
src > main > java > com > cognizant > ormlearn > J OrmLearnApplication.java > OrmLearnApplication > main(String[])
1 package com.cognizant.ormlearn;
2 import java.util.List;
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.boot.SpringApplication;
6 import org.springframework.boot.autoconfigure.SpringBootApplication;
7 import org.springframework.context.ApplicationContext;
8 import com.cognizant.ormlearn.model.Country;
9 import com.cognizant.ormlearn.service.CountryService;
10
11 @SpringBootApplication
12 public class OrmLearnApplication {
13
14     private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
15
16     private static CountryService countryService;
17
18     Run | Debug
19     public static void main(String[] args) {
20
21         ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);
22
23         countryService = context.getBean(CountryService.class);
24
25         testGetAllCountries();
26     }
27
28     private static void testGetAllCountries() {
29
30         LOGGER.info("Start");
31
32         List<Country> countries = countryService.getAllCountries();
33
34         LOGGER.debug("Countries={}", countries);
35
36         LOGGER.info("End");
37     }
38 }
```



```
src > main > java > com > cognizant > ormlearn > model > J Country.java > {} com.cognizant.ormlearn.model
1 package com.cognizant.ormlearn.model;
2 import jakarta.persistence.Column;
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.Id;
5 import jakarta.persistence.Table;
6
7 @Entity
8 @Table(name = "country")
9 public class Country {
10
11     @Id
12     @Column(name = "co_code")
13     private String code;
14
15     @Column(name = "co_name")
16     private String name;
17
18     public Country() {
19
20     }
21
22     public Country(String code, String name) {
23         this.code = code;
24         this.name = name;
25     }
26
27     public String getCode() {
28         return code;
29     }
30
31     public void setCode(String code) {
32         this.code = code;
33     }
34
35     public String getName() {
36         return name;
37     }
38 }
```

```

37
38     public void setName(String name) {
39         this.name = name;
40     }
41
42     @Override
43     public String toString() {
44         return "Country [code=" + code + ", name=" + name + "];
45     }
46 }

```

OUTPUT

```

2026-07-07 15:15:12,701} main          INFO s.b.w.e.t.TomcatWebServer Tomcat started on port 8080 (http
) with context path '/'
2026-07-07 15:15:12,709} main          INFO c.c.o.OrmLearnApplication Started OrmLearnApplication in 3.
6 seconds (process running for 3.889)
2026-07-07 15:15:12,713} main          INFO c.c.o.OrmLearnApplication Start
2026-07-07 15:15:12,842} main          DEBUG org.hibernate.SQL      select c1_0.co_code,c1_0.co_name
from country c1_0
2026-07-07 15:15:12,880} main          DEBUG c.c.o.OrmLearnApplication Countries=[Country [code=IN, name
=India], Country [code=US, name=United States of America]]
2026-07-07 15:15:12,880} main          INFO c.c.o.OrmLearnApplication End

```

Exercise 4: Difference between JPA, Hibernate and Spring Data JPA

Introduction

JPA, Hibernate, and Spring Data JPA are technologies used for database persistence in Java applications. Although they are closely related, each serves a different purpose.

Java Persistence API (JPA)

- JPA stands for **Java Persistence API**.
- It is a Java specification (JSR 338) for managing relational data.
- It defines interfaces and annotations for object-relational mapping (ORM).
- JPA itself is **not an implementation**.
- It requires an implementation such as Hibernate.

Features

- Standard API for persistence.
- Database-independent.
- Uses annotations like @Entity, @Table, @Id, etc.
- Requires an implementation provider.

Hibernate

- Hibernate is an **ORM framework**.
- It is one of the most popular implementations of JPA.
- It converts Java objects into database tables automatically.
- Provides advanced ORM features beyond JPA.

Features

- Implements JPA.
- Automatic table mapping.
- HQL (Hibernate Query Language).
- Caching support.
- Lazy loading.
- Transaction management.

Spring Data JPA

- Spring Data JPA is a Spring Framework project.
- It is built on top of JPA.
- It reduces boilerplate code by providing ready-made repository interfaces.
- Developers rarely need to write SQL queries for common operations.

Features

- Built on top of JPA.
- Uses Hibernate internally by default.
- Repository interfaces (JpaRepository).
- Automatic CRUD methods.
- Pagination and sorting.
- Transaction management using Spring.

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	Framework	Spring Module
Implementation	No	Yes	Uses JPA
ORM Support	Defines API	Provides ORM	Uses Hibernate/JPA
CRUD Operations	Manual	Manual	Automatic
Boilerplate Code	Medium	Medium	Very Low
Repository Support	No	No	Yes (<code>JpaRepository</code>)
Query Language	JPQL	HQL & JPQL	JPQL, Derived Queries
Transaction Management	Basic	Yes	Spring Managed

Hibernate Example

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();
```

```
session.save(employee);
```

```
tx.commit();
session.close();
```

Spring Data JPA Example

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}
```

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional
    public void addEmployee(Employee employee) {
        employeeRepository.save(employee);
    }
}
```

Conclusion

- **JPA** is a specification.
- **Hibernate** is an implementation of JPA that provides ORM functionality.
- **Spring Data JPA** builds on JPA and Hibernate to simplify database operations by reducing boilerplate code and providing repository-based access.